

Câu 1.Tên file: **STACK.CPP**

Để thực hành kiểu dữ liệu ngăn xếp, thầy Đạt yêu cầu học sinh bắt đầu từ một ngăn xếp rỗng và thực hiện lần lượt các yêu cầu. Yêu cầu có 4 loại như sau:

- **1** x thêm phần tử x vào ngăn xếp.
- **2** in ra độ lớn ngăn xếp và phần tử trên cùng của ngăn xếp rồi xuống dòng, nếu ngăn xếp rỗng thì in ra -1 và xuống dòng.
- **3** xoá phần tử trên cùng của ngăn xếp, nếu ngăn xếp rỗng thì in ra -1 và xuống dòng.
- **4** in ra ngăn xếp hiện có trên 1 dòng với thứ tự từ đáy ngăn xếp đến đỉnh (nếu nó rỗng thì in ra -1), sau đó kết thúc chương trình.

Thầy Đạt yêu cầu các bạn thực hiện n yêu cầu như trên.

Yêu cầu: Hãy thực hiện các yêu cầu trên.

INPUT là **STACK.INP**, gồm nhiều dòng: dòng 1 chứa số nguyên dương n ($n \leq 100$), n dòng tiếp theo chứa các yêu cầu cần thực hiện.

OUTPUT là **STACK.OUT**, gồm nhiều dòng chứa các kết quả được in ra.

EXAMPLE:

STACK.INP	STACK.OUT
5	2 4
1 3	3
1 4	
2	
3	
4	

Câu 2.Tên file: **BRACKET.CPP**

Dãy ngoặc là một dãy kí tự gồm các dấu ngoặc $(,), [,], \{, \}$ chia làm 3 cặp khớp với nhau $(,), [,], \{, \}$. Chúng ta định nghĩa dãy ngoặc **chuẩn** và **độ phức tạp** của nó theo cách đệ quy như sau:

- Dãy rỗng là dãy ngoặc **chuẩn** có **độ phức tạp** 0.
- Nếu A là dãy ngoặc **chuẩn** có **độ phức tạp** k thì (A) , $[A]$, $\{A\}$ là các dãy ngoặc **chuẩn** có **độ phức tạp** lần lượt là $k + 1$, $k + 2$ và $k + 3$.
- Nếu A và B lần lượt là dãy ngoặc **chuẩn** có **độ phức tạp** k và q thì $A + B$ hoặc $B + A$ (phép cộng hai xâu) là dãy ngoặc **chuẩn** có **độ phức tạp** $k + q$.

Cho một dãy ngoặc bất kì có độ dài không quá 10^6 kí tự.

Yêu cầu: Xác định tính **chuẩn** và **độ phức tạp** của dãy.

INPUT là **BRACKET.INP**, gồm 1 dòng chứa dãy ngoặc.

OUTPUT là **BRACKET.OUT**, gồm 1 dòng chứa YES hoặc NO để thể hiện tính chuẩn, nếu là dãy ngoặc **CHUẨN** thì in ra **độ phức tạp** trên cùng 1 dòng.

EXAMPLE:

BRACKET.INP	BRACKET.OUT
{{([[]])}}	YES 8
([])	NO

GIẢI THÍCH TEST: ở test VD1: $[] = 2 \Rightarrow ([]) = 2+1 = 3 \Rightarrow [[[]]] = 3+2 = 5 \Rightarrow \{[([[]])]\} = 8$. ($s = k$ tức là dãy ngoặc s là **chuẩn** và có **độ phức tạp** là k .)

Câu 3.Tên file: **CALCULATE.CPP**

Cho phép toán độ dài tối đa 1000 kí tự tạo thành từ các số tự nhiên 1 đến 9 xen kẽ với các dấu cộng (+), trừ (-), nhân(*), chia(/). Ta thực hiện tính toán như bình thường: nhân chia trước, cộng trừ sau.

Yêu cầu: Tính kết quả của phép toán đó.

INPUT là **CALCULATE.INP**, gồm 1 dòng chứa phép toán

OUTPUT là **CALCULATE.OUT**, gồm 1 dòng chứa kết quả của phép toán.

EXAMPLE:

CALCULATE.INP	CALCULATE.OUT
1+2*3/4	2
2-2*4+1/3	-6

Câu 4.Tên file: **INFIX.CPP**

Biểu thức toán hay được biểu diễn trong tin học là các biểu thức dạng trung tố, VD $(3 * (4 - 2) + 1) * (3 + 1) / 2$, tức là toán tử (dấu) đứng giữa hai toán hạng (số). Nhưng để máy tính tính được biểu thức của phép tính trên thì ta cần chuyển nó sang dạng tiền tố (prefix) hoặc hậu tố (postfix). Biểu thức hậu tố được hiểu là việc viết toán tử nằm sau hai toán hạng, VD $(3 * (4 - 2) + 1) * (3 + 1) / 2$ chuyển sang hậu tố sẽ là $342 - *1 + 31 + *2/$.

Yêu cầu: Chuyển từ biểu thức trung tố sang biểu thức hậu tố.

INPUT là **INFIX.INP**, gồm 1 dòng chứa biểu thức dạng trung tố dài không quá 1000 kí tự.

OUTPUT là **INFIX.OUT**, gồm 1 dòng chứa biểu thức chuyển đổi sang dạng hậu tố

EXAMPLE:

INFIX.INP	INFIX.OUT
$(3 * (4 - 2) + 1) * (3 + 1) / 2$	$342 - *1 + 31 + *2/$

Câu 5.Tên file: **POSTFIX.CPP**

Yêu cầu: Tính giá trị của biểu thức dạng hậu tố (các toán hạng là các số tự nhiên từ 1 đến 9).

INPUT là **POSTFIX.INP**, gồm 1 dòng chứa biểu thức dạng hậu tố.

OUTPUT là **POSTFIX.OUT**, gồm 1 dòng chứa kết quả biểu thức

EXAMPLE:

POSTFIX.INP	POSTFIX.OUT
$342 - *1 + 31 + *2/$	14